



GENERALITAT
VALENCIANA

Conselleria d'Educació,
Cultura i Esport



UNIÓ EUROPEA

Fons Social Europeu
L'FSE inverteix en el teu futur

IES ENRIC SOLER I GODES

Projecte d'Administració de Sistemes Informàtics en Xarxa

Curs: 2025-2026

SISTEMAS DE MONITORIZACIÓN

Autor: Kevin Murciano Gadea | Sergio Ferrá Boix

NIA: 10788620 | 10663813

Data: 23/03/26

Tutoria de projecte:

Índice

1. Introducción	3
2. Objetivos	4
3. Análisis de requerimientos	4
3.1 Necesidades, requerimientos del cliente y análisis de la situación actual	4
3.2 Estudio de alternativas de solución	5
3.3 Elección, valoración económica y diseño de las posibles soluciones	5
4. Implementación	6
4.1 Diagramas y Esquemas:	6
4.2 Herramientas de software	8
El proyecto ha utilizado exclusivamente herramientas de código abierto. No se ha adquirido ninguna licencia de software de pago.	8
4.3 Herramientas de hardware	8
4.4 Lenguajes	9
4.5 Codificación:	9
4.6 Administración	11
4.7. Aspectos de seguridad	12
5. Fases en la realització del projecte	12
6. Ampliacions	12
7. Conclusions	12
8. Bibliografia	13
9. PFC en suport físic	13
10. Annex I: Pla d'Empresa	13

1. Introducció

Hoy en día, las empresas dependen totalmente de sus ordenadores y sistemas para trabajar. Si un servidor se rompe o internet deja de funcionar, la actividad se detiene y se pierde dinero. Por eso, es fundamental tener "ojos" que vigilen todo lo que pasa en la red las 24 horas del día. A esto lo llamamos Monitorización.

Este proyecto consiste en la creación de un centro de control desde cero. Para ello, no nos hemos limitado a instalar un programa, sino que hemos preparado un servidor físico real y lo hemos dividido en varios compartimentos virtuales gracias a una herramienta llamada Proxmox.

Dentro de estos compartimentos, hemos instalado distintos programas profesionales (como Uptime Kuma o Pandora FMS) que se encargan de avisarnos si algo falla. El objetivo final es pasar de un modelo donde "arreglamos las cosas cuando se rompen" a uno donde "sabemos que algo va a fallar antes de que ocurra", garantizando que todo funcione de forma fluida y segura.

Para llevar a cabo este proyecto hemos necesitado conocimientos adquiridos en varios módulos del ciclo. Del módulo de Sistemas Operativos utilizamos los conceptos de virtualización: durante el curso instalamos y configuramos servidores, y aunque el entorno Proxmox que usamos en el proyecto fue preparado previamente por el profesor, la experiencia con esa herramienta nos permitió movernos con soltura dentro de él. Del módulo de Redes aplicamos los fundamentos de comunicación entre sistemas y los conocimientos sobre Docker, que nos sirvieron de base para entender cómo desplegar servicios en contenedores dentro de la infraestructura.

2. Objetivos

El objetivo principal de este trabajo es desplegar un sistema de monitorización completo que vigile automáticamente el estado de los equipos y servicios informáticos, garantizando que estén siempre disponibles y que cualquier problema sea detectado y notificado antes de que afecte al funcionamiento.

Los objetivos específicos son los siguientes:

- Montar y configurar un servidor físico real sobre el que desplegar toda la infraestructura de virtualización, utilizando Proxmox VE como plataforma base.
- Instalar y configurar herramientas de monitorización profesionales (Uptime Kuma y Pandora FMS) capaces de supervisar el estado de servicios y equipos en tiempo real.
- Implementar un sistema de alertas que notifique automáticamente al equipo técnico cuando se detecte un fallo o una anomalía, mediante Gotify.
- Garantizar la seguridad del entorno configurando accesos restringidos y copias de seguridad automáticas para proteger la información ante cualquier incidente.
- Validar el funcionamiento del sistema simulando fallos reales y comprobando que la detección y notificación se producen correctamente.
- Documentar todo el proceso de instalación y configuración para que cualquier persona pueda mantener o reproducir el sistema en el futuro.

3. Análisis de requerimientos

3.1 Necesidades, requerimientos del cliente y análisis de la situación actual

El punto de partida es una infraestructura sin ningún sistema de monitorización centralizado. Los equipos son gestionados de forma reactiva: las incidencias se detectan únicamente cuando los usuarios las reportan, lo que conlleva tiempos de resolución elevados y pérdida de productividad.

Los requerimientos principales identificados son: (a) supervisión continua del estado de las máquinas virtuales y los servicios web, (b) notificaciones inmediatas ante caídas o degradaciones de rendimiento, y (c) registro histórico de métricas para análisis posterior.

3.2 Estudio de alternativas de solución

Para el diseño de la infraestructura, se han evaluado diferentes grupos de soluciones técnicas para cada componente crítico de la arquitectura, analizando su viabilidad, costes y facilidad de implementación:

Virtualización: Las alternativas consideradas fueron VMware ESXi (solución propietaria, licencia de coste elevado), Hyper-V (integrado en Windows Server, requiere licencia) y Proxmox VE (código abierto, basado en Debian Linux y KVM).

Se optó por **Proxmox VE** por su coste cero, su comunidad activa y su interfaz web de administración completa.

Monitorización de disponibilidad: Se compararon Nagios (potente pero complejo de configurar), Zabbix (solución completa pero con alta curva de aprendizaje) y Uptime Kuma (interfaz moderna, fácil despliegue en Docker).

Se eligió **Uptime Kuma** por su sencillez de configuración y su integración nativa con múltiples canales de alerta.

Monitorización avanzada de rendimiento: Se valoraron Zabbix, Checkmk y Pandora FMS.

Se seleccionó **Pandora FMS** por su versión comunitaria gratuita, su soporte de agentes SNMP y la capacidad de generar informes detallados.

Sistema de notificaciones: Las alternativas analizadas fueron Telegram Bot (dependencia de terceros), ntfy (autoalojado, más básico) y Gotify (autoalojado, aplicación Android oficial, API REST sencilla).

Se eligió **Gotify y ntfy** por su total independencia de servicios externos y su integración con Uptime Kuma.

3.3 Elección, valoración económica y diseño de las posibles soluciones

El coste de implantación de este proyecto es prácticamente nulo gracias a dos factores clave: la donación del hardware y el uso exclusivo de software de código abierto.

Hardware El servidor físico sobre el que se ejecuta Proxmox VE ha sido donado por una de las empresas en las que los autores realizan el módulo de Formación en Centros de Trabajo (FCT). Esta cesión elimina por completo el principal coste de cualquier infraestructura de virtualización.

No obstante, para una implementación desde cero, un servidor con las características necesarias para este proyecto (procesador de gama media-profesional, 32GB de RAM y almacenamiento redundante SSD) tendría un **coste de mercado estimado de entre 800 € y 1.200 €**.

Software Todas las herramientas utilizadas en el proyecto son gratuitas y de código abierto:

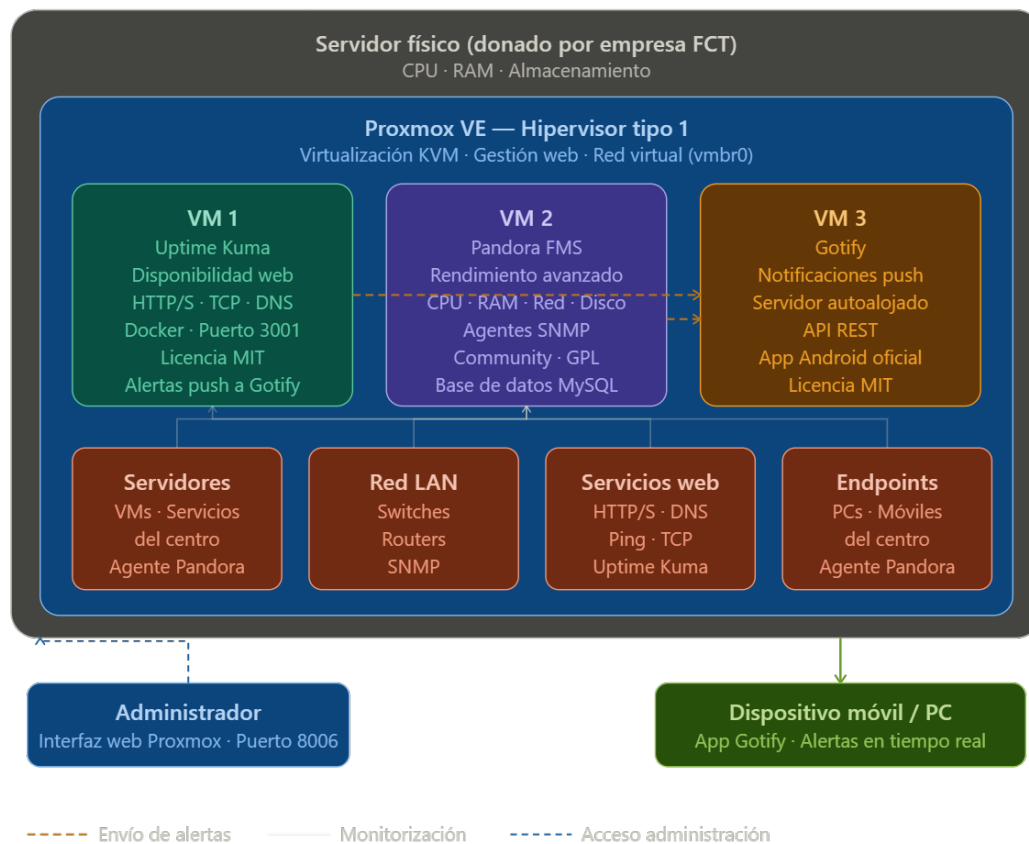
- **Proxmox VE:** hipervisor de tipo 1 publicado bajo licencia AGPL. Su descarga e instalación son completamente gratuitas.
- **Uptime Kuma:** publicado bajo licencia MIT. No existe versión de pago; toda la funcionalidad está disponible sin coste.
- **Gotify:** publicado bajo licencia MIT. Servidor de notificaciones autoalojado sin ningún modelo de suscripción.
- **Pandora FMS Community:** se utiliza la edición Community (licencia GPL). Cabe destacar que, aunque existe una versión Enterprise de pago con soporte oficial y módulos avanzados, la versión gratuita cubre todas las necesidades de este análisis.

Componente	Solución elegida	Coste Real	Coste Estimado (Compra)
Servidor físico	Donado por empresa FCT	0€.	~1000€.
Hipervisor	Proxmox VE (AGPL)	0€.	0€.
Monitorización disponibilidad	Uptime Kuma (MIT)	0€.	0€.
Notificaciones push	Gotify (MIT)	0€.	0€.
Monitorización rendimiento	Pandora FMS Community	0€.	0€.
TOTAL		0€.	~1000€.

4. Implementación

En este apartado se detalla el despliegue técnico del sistema. La solución propuesta no es una elección arbitraria, sino la respuesta directa a las necesidades identificadas: se ha optado por un **entorno virtualizado con Proxmox VE** para garantizar flexibilidad y aislamiento, el uso exclusivo de **herramientas de código abierto** para eliminar costes de licencias, y una arquitectura de **servicios independientes** (Uptime Kuma, Pandora FMS y Gotify) para cubrir desde la disponibilidad inmediata hasta el análisis de rendimiento avanzado y la gestión de alertas sin dependencias externas.

4.1 Diagramas y Esquemas:



A continuación se presenta el esquema general de la arquitectura del sistema de monitorización implementado. El proyecto se estructura en tres capas principales: el hardware físico, la capa de virtualización gestionada por Proxmox VE, y los servicios de monitorización desplegados en máquinas virtuales independientes.

La figura 1 muestra cómo el servidor físico aloja el hipervisor Proxmox VE, que a su vez gestiona tres máquinas virtuales: una para **Uptime Kuma** (monitorización de disponibilidad), otra para **Pandora FMS** (monitorización avanzada de rendimiento) y una tercera para **Gotify** (servidor de notificaciones push autoalojado). Uptime Kuma y Pandora FMS envían alertas a Gotify, que distribuye las notificaciones push a los dispositivos móviles y ordenadores de los administradores.

Component	Funció principal
Servidor físico (hardware) curl -SSL https://pfms.me/de	Base de cómputo: CPU, RAM y almacenamiento para todas las VMs.
Proxmox VE (hipervisor)	Virtualización de tipo 1 basada en KVM. Gestión de VMs desde interfaz web.
VM 1 — Uptime Kuma	Monitorización de disponibilidad de servicios web y endpoints (HTTP/S, TCP, DNS).
VM 2 — Pandora FMS Community	Monitorización avanzada de rendimiento: CPU, RAM, red, disco. Agentes SNMP.
VM 3 — Gotify	Servidor de notificaciones push autoalojado. App Android oficial. API REST.

4.2 Herramientas de software

El proyecto ha utilizado exclusivamente herramientas de código abierto. No se ha adquirido ninguna licencia de software de pago.

- **Proxmox VE 8.x:** Hipervisor de tipo 1 basado en Debian 12 y KVM. Permite la gestión de VMs y contenedores LXC mediante interfaz web (Licencia AGPL).
- **Ubuntu Server 24.04:** Sistema operativo base de las máquinas virtuales donde se instalan los servicios de Gotify y Uptime Kuma.
- **Almalinux 9:** Sistema operativo recomendado para desplegar Pandora FMS.
- **Uptime Kuma 1.x:** Aplicación de monitorización de tiempo de actividad desplegada en Docker. Soporta HTTP/S, TCP, ping, DNS y SNMP (Licencia MIT).
- **Docker Engine:** Plataforma de contenedores utilizada para el despliegue de Uptime Kuma.
- **Pandora FMS Community Edition:** Plataforma de monitorización empresarial en versión comunitaria gratuita. Supervisa CPU, RAM, disco y red mediante agentes (Licencia GPL).
- **MySQL / MariaDB:** Base de datos utilizada por Pandora FMS para almacenar métricas y configuración.
- **Gotify:** Servidor de notificaciones push autoalojado con API REST y aplicación oficial para Android (Licencia MIT).
- **Otras herramientas:** Git para control de versiones, cURL/wget para pruebas de conectividad y nano/vim como editores de texto.

Revisión del servidor físico: Verificación de los componentes (CPU, RAM y Almacenamiento) para asegurar que soportarían la carga de tres máquinas virtuales simultáneas.

4.3 Herramientas de hardware

El servidor físico es el elemento central de la infraestructura. Fue donado por una de las empresas donde los autores realizan el módulo de Formación en Centros de Trabajo (FCT).

Componente de hardware	Detalle / Configuración
Procesador (CPU)	Arquitectura x86-64 con soporte de virtualización (Intel VT-x / AMD-V).
Memoria RAM	Capacidad suficiente para alojar Proxmox VE y tres máquinas virtuales simultáneas.
Almacenamiento	Disco duro o SSD local. Proxmox gestiona el pool (LVM o ZFS).
Interfaz de red	Tarjeta Ethernet integrada. Proxmox crea un bridge virtual (vbr0) para la LAN.
Conexión a la red	El servidor se conecta a la LAN del centro, permitiendo acceso remoto a la interfaz web.

Configuración del hardware: Se verificaron físicamente los componentes, se activó la virtualización en la BIOS/UEFI y se instaló Proxmox VE mediante USB, configurando una IP estática y el bridge de red virtual.

4.4 Lenguajes

El proyecto se centra en la administración de sistemas, pero se han empleado diversos lenguajes y formatos para la automatización y configuración:

- **Bash / Shell scripting:** Para automatizar la instalación de paquetes, arranque de Docker y copias de seguridad.
- **YAML:** Formato utilizado para la exportación y restauración de configuraciones en Uptime Kuma.
- **JSON:** Estructura de los mensajes para la API REST de Gotify.
- **INI / ficheros de configuración:** Utilizados por Pandora FMS para definir agentes y parámetros de conexión.
- **HTML y CSS:** Utilizados internamente por las consolas web de las herramientas.

4.5 Codificación:

A continuación se muestran los scripts más relevantes. El resto de ficheros se encuentran en el soporte físico adjunto.

Script de instalación de Uptime Kuma (install_uptime_kuma.sh): Automatiza la actualización del sistema, la instalación de Docker y la ejecución del contenedor de Uptime Kuma con

persistencia de datos.

```
#!/bin/bash
# Instalación de Docker + Uptime Kuma
# Compatible con Ubuntu Server 24.04
# Ejecutar con privilegios de superusuario (root)

set -e # Finaliza el script si algún comando devuelve un error

# 1. Instalar dependencias del sistema
apt-get update -y
apt-get install -y ca-certificates curl gnupg

# 2. Configurar el repositorio oficial de Docker
install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | gpg --dearmor -o /etc/apt/keyrings/docker.gpg
chmod a+r /etc/apt/keyrings/docker.gpg

echo \
    "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] \
        https://download.docker.com/linux/ubuntu $(. /etc/os-release && echo
"$VERSION_CODENAME") stable" | \
    tee /etc/apt/sources.list.d/docker.list > /dev/null

# 3. Instalar Docker Engine y plugins
apt-get update -y
apt-get install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin

# 4. Habilitar y arrancar el servicio de Docker
systemctl enable --now docker

# 5. Desplegar contenedor de Uptime Kuma
```

```
# Se utiliza un volumen persistente para no perder la configuración al reiniciar
docker run -d \
  --restart=always \
  -p 3001:3001 \
  -v uptime-kuma:/app/data \
  --name uptime-kuma \
  louislam/uptime-kuma:1

echo ""
echo "Instalación completada con éxito."
echo "Accede a la interfaz web en: http://$(hostname -I | awk '{print $1}'):3001"
```

Script instalación de Pandora FMS (pandorafms_install.sh): Despliega pandora FMS con la versión gratuita.

```
#!/bin/bash
# Instalación de Pandora FMS - Entorno Monolítico (EL9)
# Doc oficial: https://pandorafms.com/manual/current/es/documentation/pandorafms/installation/01_installing
# Ejecutar como root en EL9 (RHEL 9 / Rocky Linux 9 / AlmaLinux 9)

curl -sS https://pfms.me/deployenterprise | \
  TZ='Europe/Madrid' \
  DBHOST='127.0.0.1' \
  DBNAME='pandora' \
  DBUSER='pandora' \
  DBPASS='tucontraseñasegura99abc!' \
  DBPORT='3306' \
  DBROOTPASS='tucontraseñasegura99abc!' \
  MYVER=80 \
  PHPVER=8 \
  SKIP_PRECHECK=0 \
  SKIP_DATABASE_INSTALL=0 \
  SKIP_KERNEL_OPTIMIZATIONS=0 \
  PANDORA_AGENT_PACKAGE="https://firefly.pandorafms.com/pandorafms/latest/pandorafms_one_agent_linux_bin-latest.el9.x86_64.rpm" \
```

```
PANDORA_BETA=0 \  
PANDORA_LTS=1 \  
PANDORA_ENT_FREE=1 \  
bash
```

Script instalación de gotify

```
#!/bin/bash  
# Instalación de Docker + Gotify  
# Compatible con Ubuntu Server 24.04  
# Ejecutar con privilegios de superusuario (root)  
  
set -e # Finaliza el script si algún comando devuelve un error  
  
# 1. Instalar dependencias del sistema  
apt-get update -y  
apt-get install -y ca-certificates curl gnupg  
  
# 2. Configurar el repositorio oficial de Docker  
install -m 0755 -d /etc/apt/keyrings  
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | gpg --dearmor -o  
/etc/apt/keyrings/docker.gpg  
chmod a+r /etc/apt/keyrings/docker.gpg  
  
echo \  
"deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] \  
https://download.docker.com/linux/ubuntu $(. /etc/os-release && echo  
"$VERSION_CODENAME") stable" | \  
tee /etc/apt/sources.list.d/docker.list > /dev/null  
  
# 3. Instalar Docker Engine y plugins  
apt-get update -y  
apt-get install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-  
plugin  
  
# 4. Habilitar y arrancar el servicio de Docker  
systemctl enable --now docker
```

```
# 5. Desplegar contenedor de Gotify
# Se utiliza un volumen persistente para guardar los mensajes y la configuración
docker run -d \
  --restart=always \
  -p 8080:80 \
  -v gotify-data:/app/data \
  --name gotify \
  gotify/server

echo ""
echo "Instalación completada con éxito."
echo "Accede a la interfaz web en: http://$(hostname -I | awk '{print $1}'):8080"
echo "Credenciales por defecto: usuario 'admin', contraseña 'admin'"
```

4.6 Administración

El sistema se gestiona a través de tres interfaces web independientes:

- **Proxmox VE:** Puerto 8006 (Gestión global).
- **Uptime Kuma:** Puerto 3001 (Panel de monitores).
- **Pandora FMS:** /pandora_console (Consola completa).
- **Gotify:** Puerto 80 (Gestión de tokens).

Se han definido perfiles de usuario como **root/admin** para acceso total y perfiles de **monitorización/operador** para tareas de solo lectura, garantizando la seguridad y trazabilidad.

4.7. Aspectos de seguridad

Para garantizar la integridad del sistema y la protección de los datos de monitorización recopilados, se han implementado una serie de medidas y protocolos de seguridad que blindan la infraestructura frente a posibles amenazas externas e internas:

- **Prevención de accesos:** Uso de contraseñas robustas (mínimo 12 caracteres) y restricción de acceso a las consolas web únicamente desde la LAN del centro; ningún puerto es accesible desde Internet.
- **Aislamiento y Actualización:** Cada servicio corre en una VM independiente para contener posibles fallos. Se mantienen actualizados tanto el SO como los contenedores Docker.

- **Protección de aplicaciones:** Las herramientas incluyen protección nativa contra fuerza bruta y vulnerabilidades web como XSS o inyección SQL.
- **Recuperación:** Se realizan copias de seguridad periódicas mediante *cron jobs* que incluyen snapshots de Proxmox, volcados de base de datos MySQL (Pandora) y copias de directorios de datos (Uptime Kuma y Gotify).

5. Fases en la realització del projecte

El proyecto se ha desarrollado siguiendo un enfoque progresivo, desde la adecuación de la infraestructura base hasta la validación final del sistema de alertas.

Fase 1 — Adecuación del Entorno de Virtualización

Duración: Enero – Febrero

En esta fase se preparó el entorno lógico sobre el que se despliega el proyecto. Al usar un servidor físico ya montado en el centro, los esfuerzos se centraron en la gestión del hipervisor Proxmox.

1. **Revisión del servidor físico:** Verificación de los componentes (CPU, RAM y almacenamiento) para asegurar que soportarían la carga de las tres máquinas virtuales simultáneas.
2. **Gestión del Hipervisor:** Acceso y configuración de Proxmox VE 8.x. Se establecieron las interfaces de red mediante un bridge virtual (vmbr0) para dar conectividad a los servicios.
3. **Despliegue de Máquinas Virtuales (VM):** Creación de tres entornos independientes basados en Ubuntu Server 24.04 y Almalinux 9, optimizando los recursos para cada servicio.

Fase 2 — Instalación y Configuración de Servicios

Duración: Febrero – Abril

Se procedió a la instalación de las herramientas de monitorización y notificaciones, garantizando el aislamiento de cada una en su propia VM.

1. **Monitorización de Disponibilidad:** Instalación de Uptime Kuma mediante contenedores Docker para supervisar servicios web y protocolos DNS/TCP.
2. **Monitorización de Rendimiento:** Despliegue de Pandora FMS Community sobre AlmaLinux para el control avanzado de métricas de hardware (CPU, RAM, Disco) mediante agentes y SNMP.
3. **Centro de Notificaciones:** Configuración de Gotify como servidor centralizado de alertas push, eliminando la dependencia de servicios externos.
4. **Integración de Alertas:** Vinculación de Uptime Kuma y Pandora FMS con la API de Gotify para que los fallos detectados se notifiquen instantáneamente a los administradores.

Fase 3 — Seguridad y Optimización

Duración: Abril – Mayo

Una vez operativos los servicios, se aplicaron medidas para garantizar la robustez del sistema y la protección de los datos.

1. **Blindaje de Red:** Restricción de acceso a las interfaces web (puertos 8006, 3001, 80, 265) únicamente desde la LAN, protegiendo el sistema de accesos externos.
2. **Políticas de Backup:** Configuración de copias de seguridad automáticas de las VMs en Proxmox y volcados de base de datos MySQL para Pandora FMS.
3. **Gestión de Credenciales:** Implementación de contraseñas robustas y perfiles de usuario con privilegios restringidos (root/admin vs. operador).

Fase 4 — Comprobaciones y Validación Final

Duración: Mayo – Junio

La fase final sirvió para asegurar que el sistema cumple con el objetivo de pasar de un modelo reactivo a uno proactivo.

1. **Simulación de Incidencias:** Se provocaron fallos controlados para validar que las herramientas detectaban el error y Gotify enviaba la notificación en tiempo real.
2. **Ajuste de Umbrales:** Optimización de las reglas de alerta para evitar falsos positivos y asegurar que la información sea útil.
3. **Elaboración de la Memoria:** Documentación técnica detallada (scripts, esquemas y configuraciones) para permitir la replicabilidad del proyecto.

6. Ampliaciones

Como ampliación del proyecto, se plantea añadir un portal web centralizado llamado KS-Dashboard, desde el cual los usuarios podrán acceder a todas las herramientas de monitorización del sistema: Pandora FMS, Uptime Kuma, Gotify y ntfy. El portal estará desarrollado en HTML y CSS y será servido mediante nginx.

Para dotarlo de alta disponibilidad, se desplegarán dos servidores nginx con Keepalived, que implementa el protocolo VRRP. Uno actuará como MASTER y el otro como BACKUP, compartiendo una IP virtual (VIP) a la que accederán los usuarios. Si el MASTER cae, el BACKUP adquiere automáticamente esa IP y continúa sirviendo el portal en menos de 3 segundos, de forma completamente transparente para el usuario.

Para mantener el contenido sincronizado entre los dos nodos se utilizará lsyncd, que detecta cambios en el directorio del dashboard en tiempo real y los replica al segundo servidor mediante rsync sobre SSH.

Desde el punto de vista de la seguridad, se configurarán reglas de firewall para permitir el protocolo VRRP exclusivamente entre los dos nodos, la autenticación entre ellos estará protegida por contraseña compartida, y la sincronización entre servidores se realizará siempre mediante claves SSH, nunca con credenciales en texto plano.

7. Conclusions

Comenta breument les impressions que t'ha produït el projecte, dificultats trobades, aspectes que més t'ha costat de resoldre, etc....

8. Bibliografia

Inclou ací els aspectes bibliogràfics de consulta i fonts d'informació: llibres, webs, manuals, referències...

Norma APA

9. PFC en suport físic

Enganxa o adjunta en la contraportada el PFC en suport físic (pendrive USB) tant el codi com els documents de memòria i presentacions, així com qualsevol contingut que servisca per a avaluar-lo i contrastar la teua autoria.

10. Annex I: Pla d'Empresa

Adjunta en el suport físic el document amb el Pla d'Empresa, si has decidit incloure'l en el projecte